



University
of Exeter

COMM117

Large Language Models and Application

Tokenization Type

Character vs word vs subword vs byte

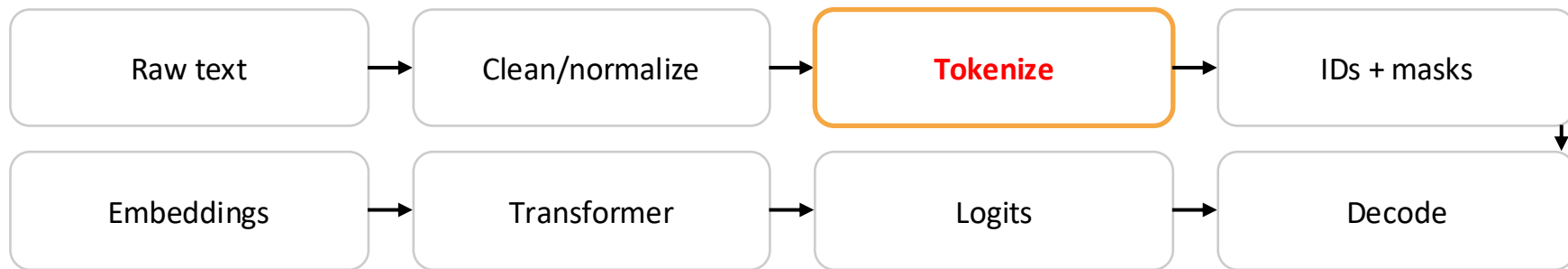
Unit	Pros	Cons	Typical use
Character	No OOV; robust	Very long sequences	Small models; some multilingual
Word	Readable; short seq.	OOV + huge vocab	Classic NLP; not common in LLMs
Subword	Good trade-off	Tokenizer quirks	Most modern LLMs
Byte	Covers any text	More tokens in some cases	Byte-level BPE family

BPE: algorithm (high-level)

- Input: training corpus + **target** vocabulary size
- Count neighbor symbol pairs across the corpus
- Merge the most frequent pair → create a new symbol
- Repeat until you reach the desired vocab size
- Tokenize new text by applying the learned merges

End-to-end NLP / LLM pipeline

Where tokenization fits

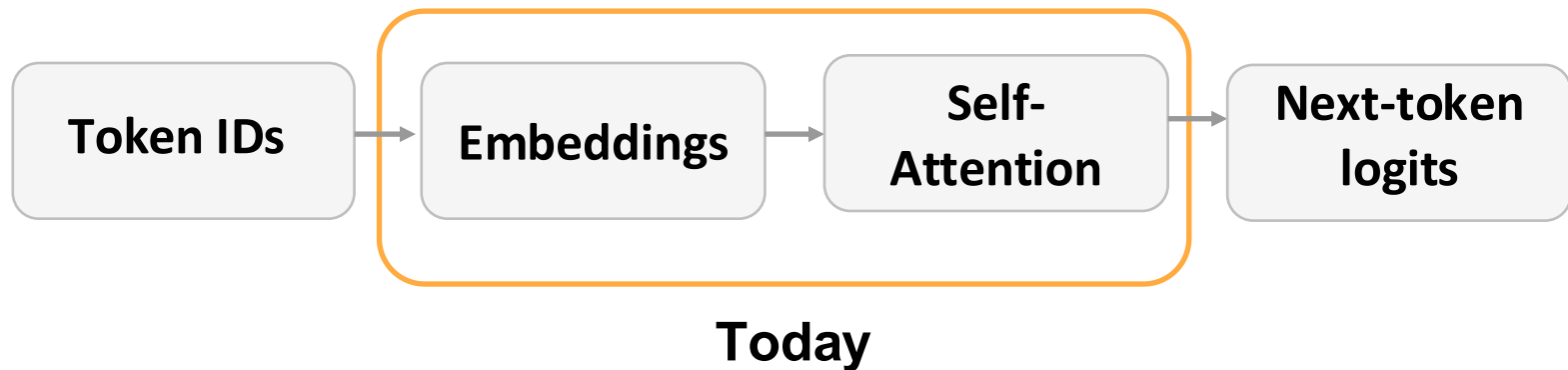


- Training: next-token prediction (teacher forcing)
- Inference: autoregressive loop with caching (KV cache)
- Tokenization/vocabulary define the model's input and output space

Introduction Today

- Embedding
- From RNN to Transformer
- What is Attention and why Attention?

From tokens to meaning



Why sequences are hard

- “**bank**” in “river bank” vs “bank loan”
- Meaning depends on **context** and order
- We need representations that **change with neighbors**

Sequences Modeling

- Language is a **mapping from sequences to meaning**

sequence modelling stands as the cornerstone of predicting the next element in a sequence of data

Sequences Modeling

- **Contextual representations**

Each token's vector should depend on surrounding tokens
("bank" changes with "river" vs "loan")

- **Long-range dependency handling**

Connect information across many positions (pronouns, topics, constraints)

- **Order awareness**

The model must distinguish "dog bites man" vs "man bites dog"

- **Variable-length inputs/outputs**

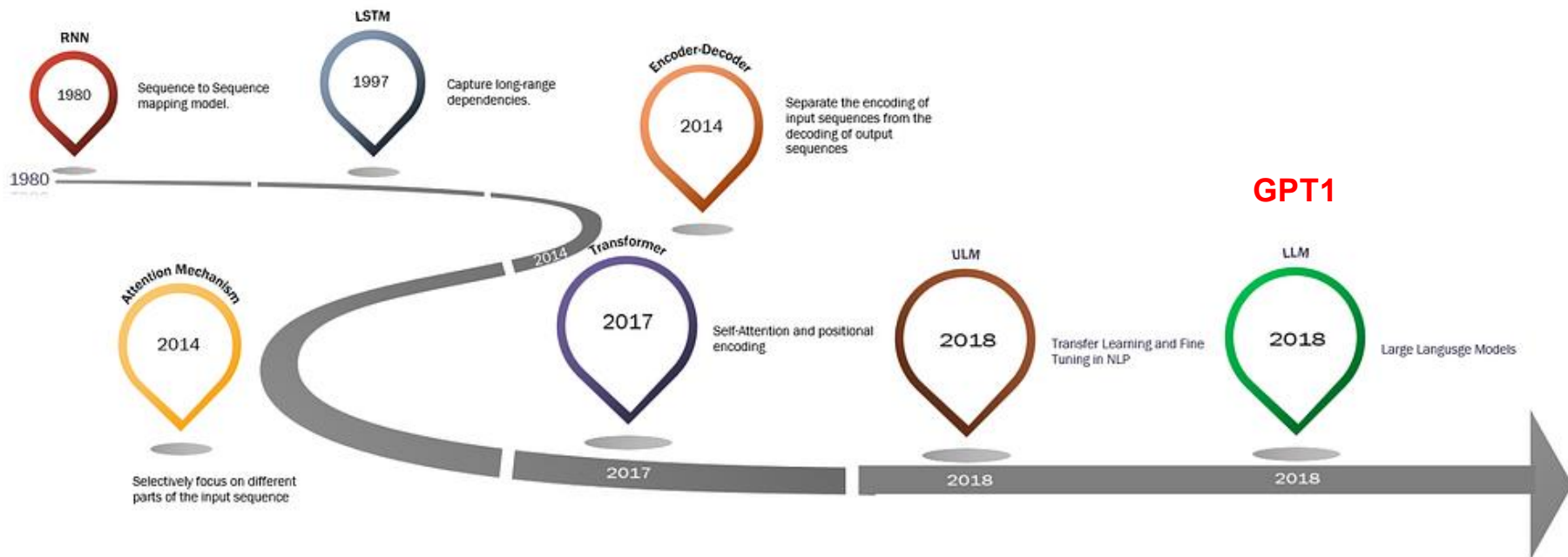
Sentences, paragraphs, conversations; generate sequences of different lengths

- **Efficient computation at scale**

Training/inference must be feasible on long sequences

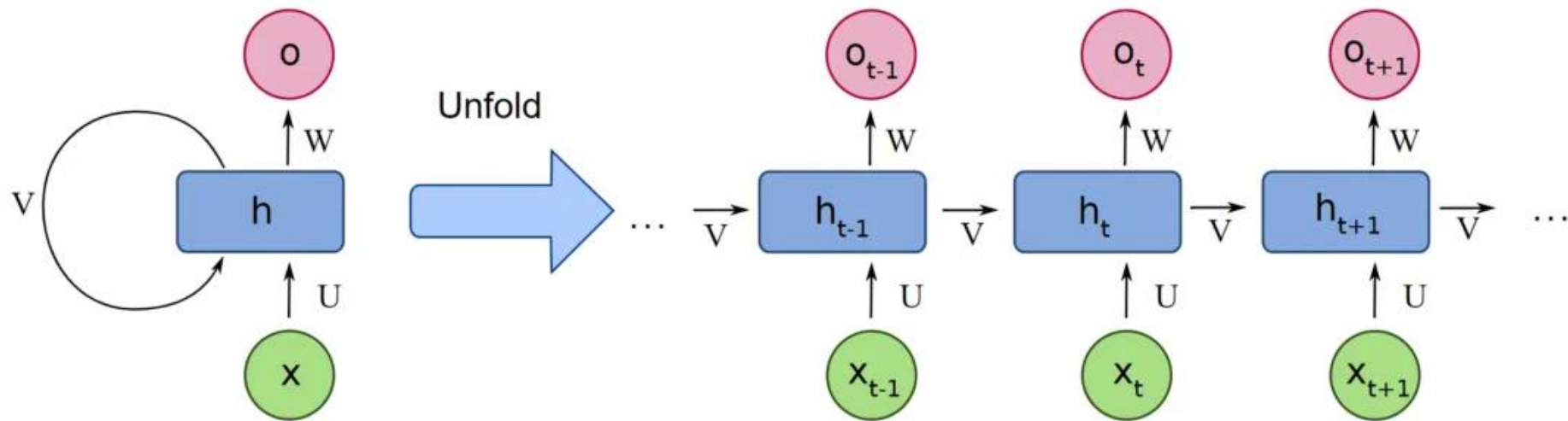
LLMs Journey

Architecture: the Transformer



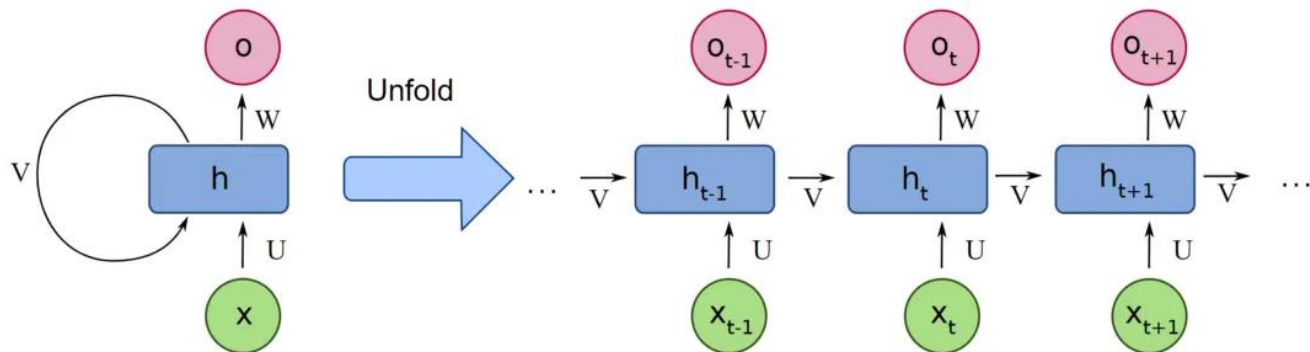
Pre-Transformer: RNNs

Recurrent Neural Networks



Pre-Transformer: RNNs

- RNN updates hidden state **sequentially**
--- Memories
- Good for **variable length** sequences



Pre-Transformer: RNNs

-
- RNN updates hidden state **sequentially**
--- Memories
 - Good for **variable length** sequences

Pre-Transformer: RNNs

But:

- **hard to parallelise**
sequential dependency
- **long-range dependencies**
the key information needed for a decision is far away

Pre-Transformer: RNNs

But:

- **hard to parallelise**
sequential dependency
- **long-range dependencies**
the key information needed for a decision is far away

Transformer!

- **Sequential compute → more parallel training**
-

RNN: must process tokens **one-by-one**.

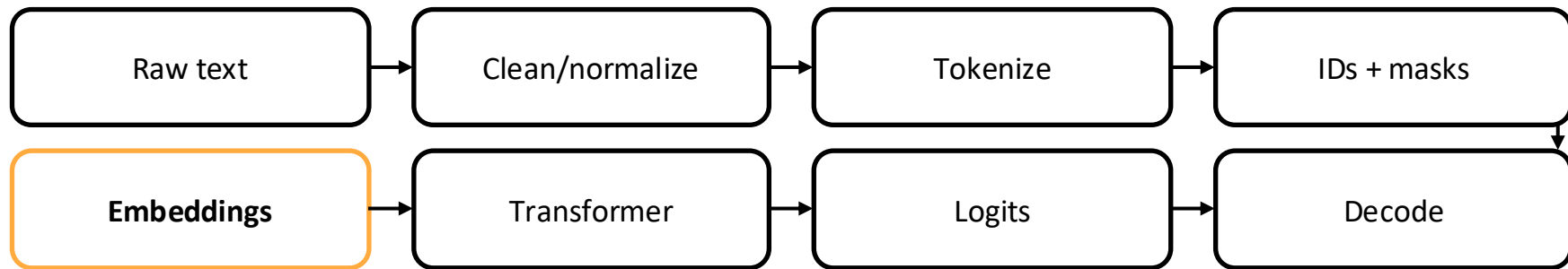
Transformer/LLM: in each layer, it computes **attention** for all positions **in parallel** (during training).

- **Long-range dependencies → direct connections via attention**

RNN: information must travel through many steps (weak signal over long distance).

LLM: a token can attend **directly to a far-away token** in one operation.

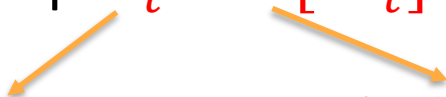
End-to-end NLP / LLM pipeline



Tokens To Embedding

- Vocabulary: the list of tokens the tokenizer/model knows.
- Token IDs: the integer index of a token in that vocabulary.
- **Embedding: a learnable vector the model looks up using the token ID.**

Embedding lookup: $e_t = E[ID_t]$ (row of the embedding matrix)


Embedding **vector** for Token ID_t embedding matrix

IDs are discrete; the model operates on vectors.

Embedding

- An **embedding** is a **learnable vector** representation of a discrete symbol
- During training, gradient descent updates these vectors like any other parameter.

Embedding

Why Embedding

- IDs are arbitrary labels (no numeric meaning)
- Embeddings make tokens learnable in a continuous space
- **Similarity in embedding space** encodes relationships
 - dot product / cosine similarity $\approx$ “related meaning/use”
- **Directions** can represent features (e.g., tense, plurality, domain)

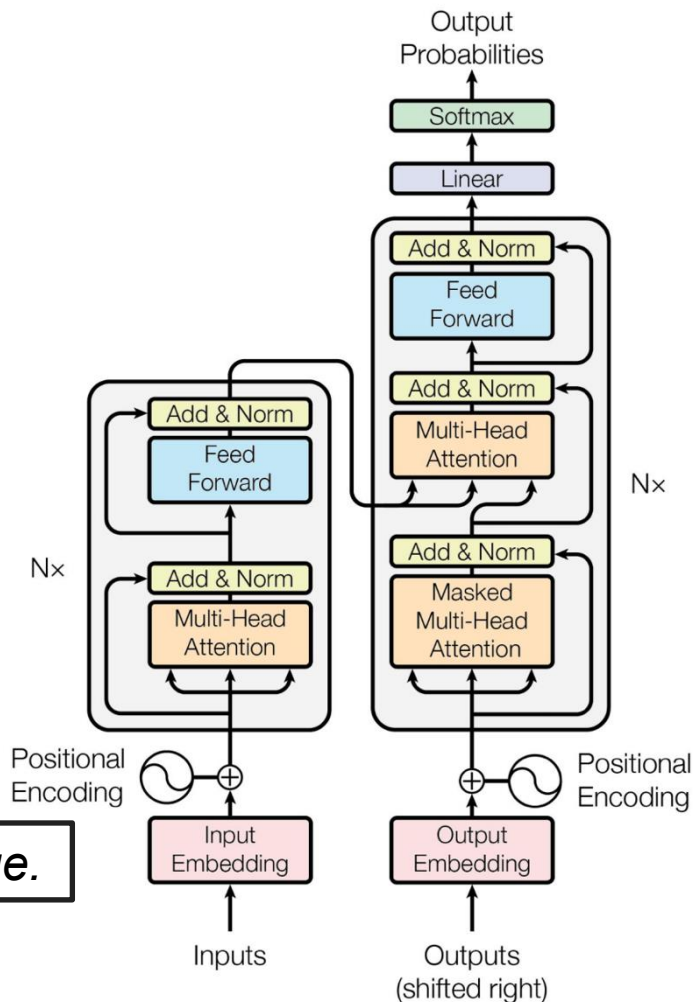
Transformer

Task: **Translator**

I studied large language models.



French: *J'ai étudié les grands modèles de langage.*

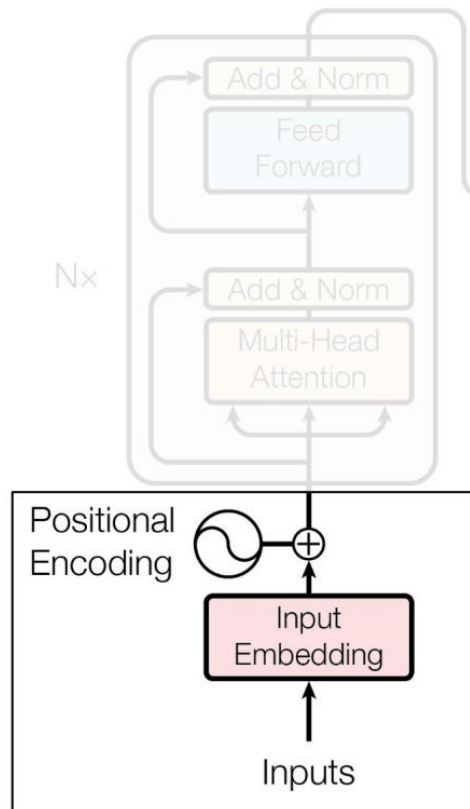


Tokenization

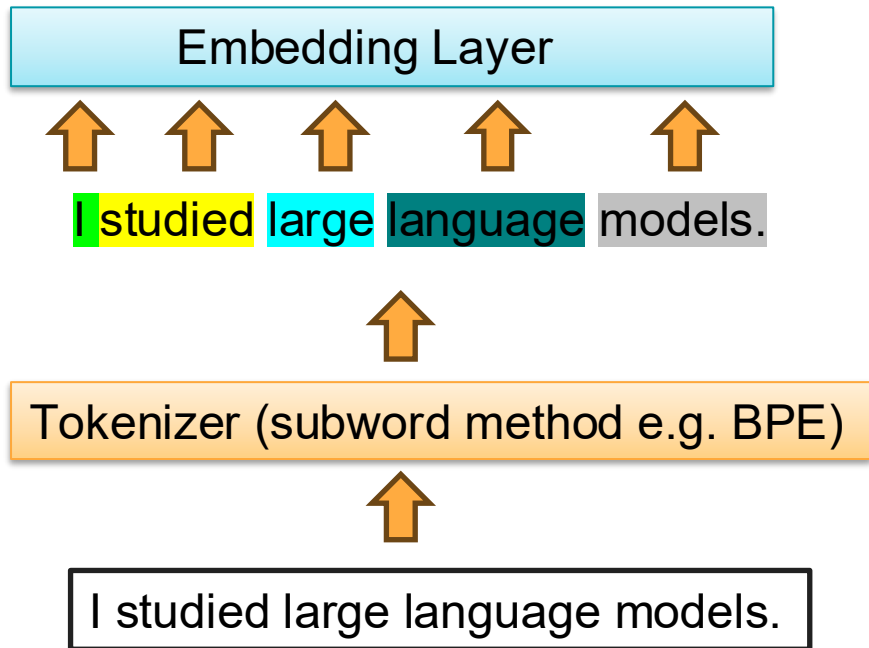
Tokenizer (subword method e.g. BPE)



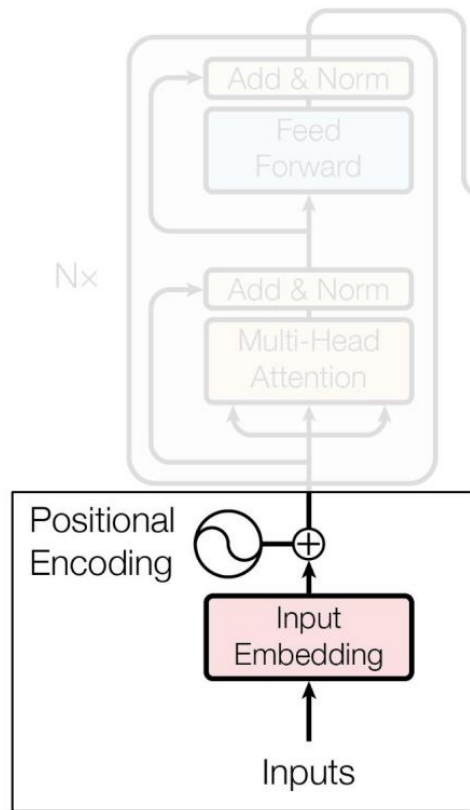
I studied large language models.



Embedding in LLM



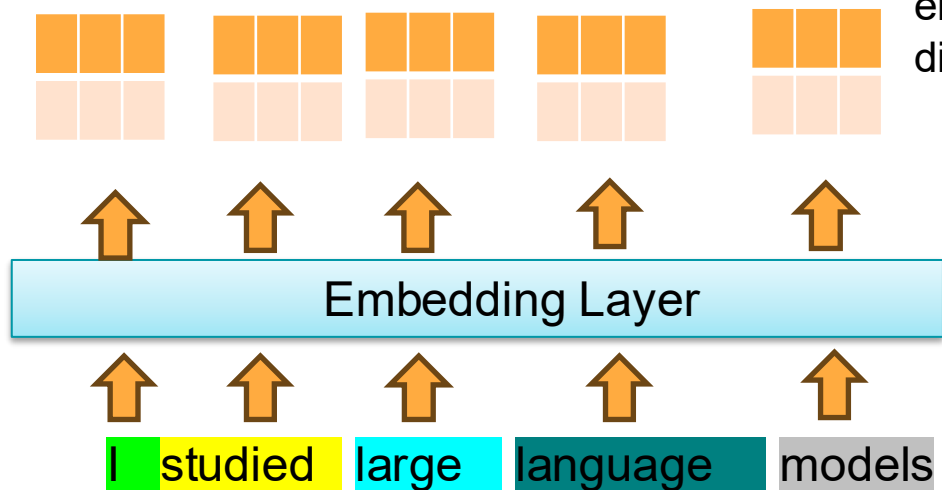
Generate Input Embeddings



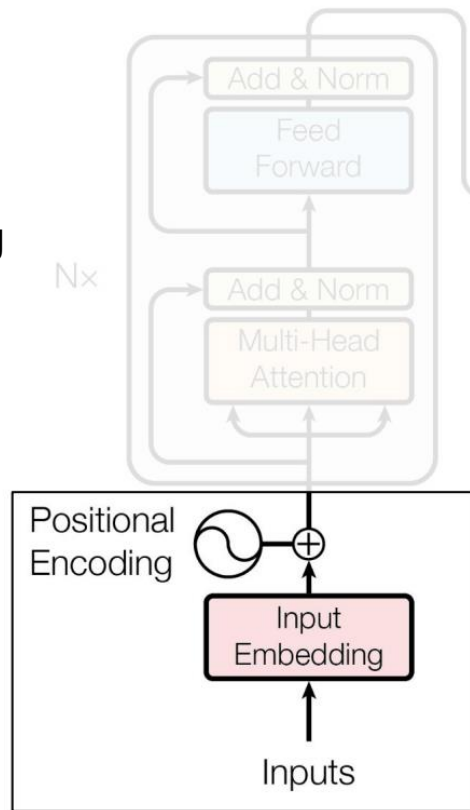
Embedding in LLM

$$E \in \mathbb{R}^{|V| \times d}$$

D is model
embedding
dimension

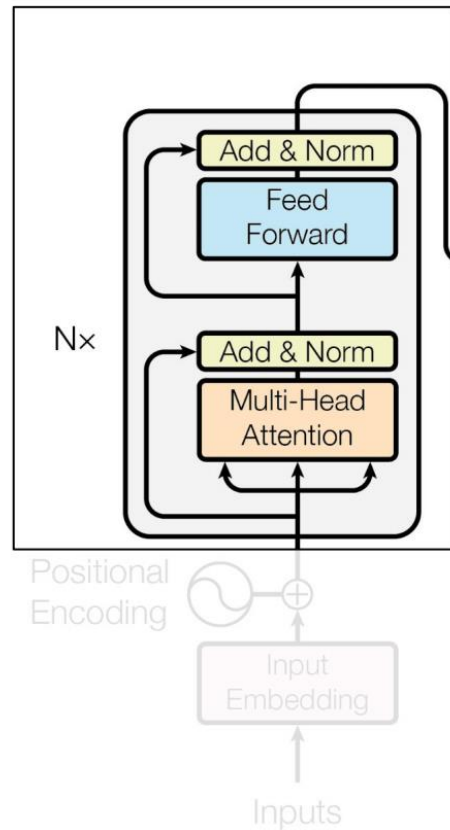
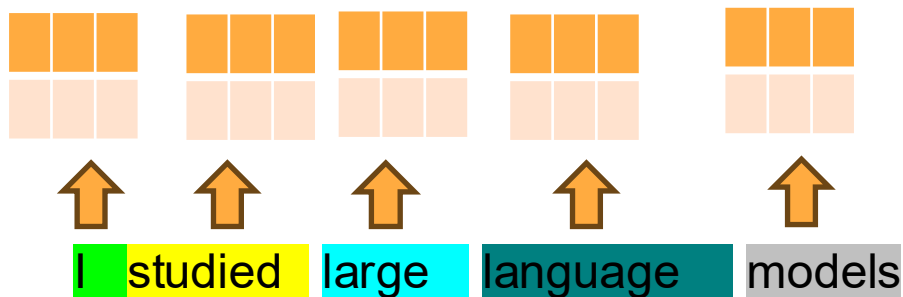


Generate Input Embeddings



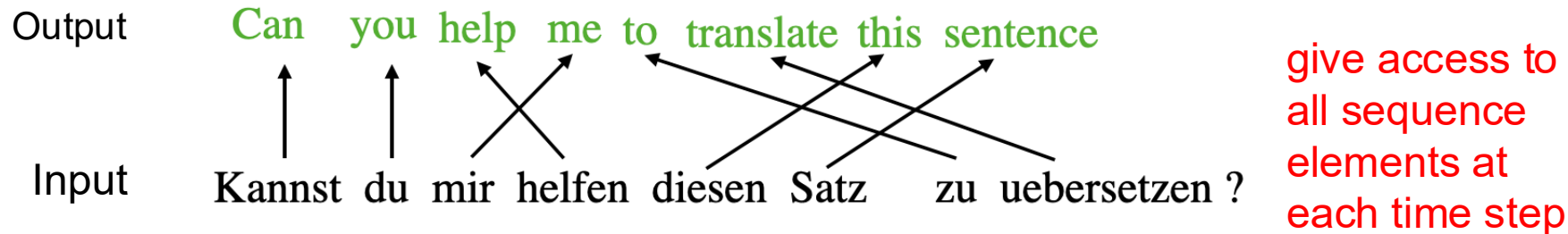
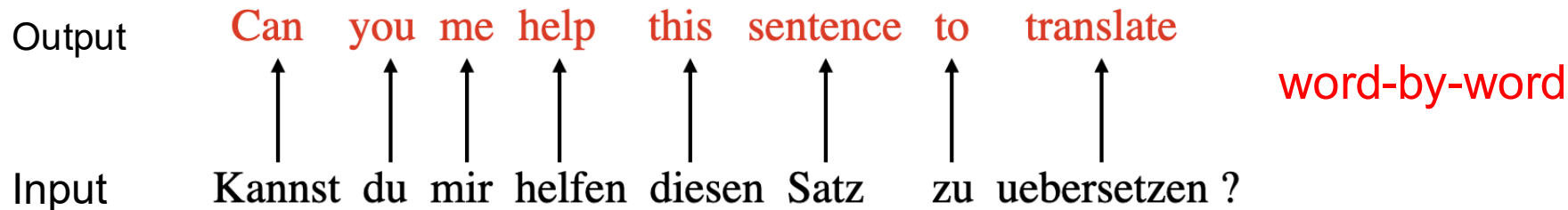
Encoder in LLM

How to build context?



Attention

Goal: improve RNN for handling longer sequences



Attention

Attention mechanisms enhance deep learning models by **selectively focusing on important input elements**



Q: How many students in the photo?

Q: Who is the teacher in the photo

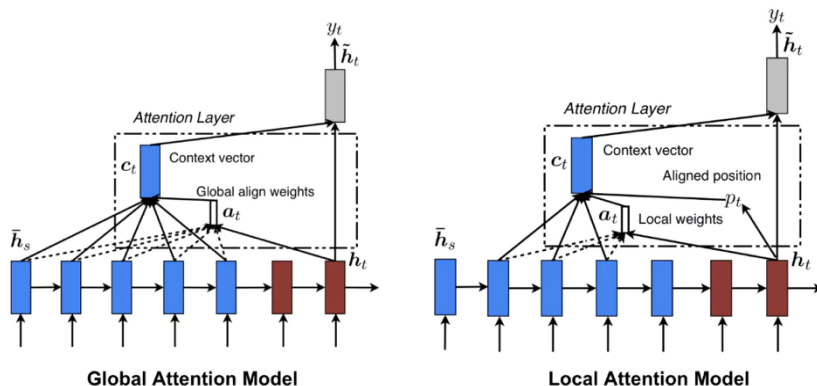
Teacher feature is the most important
Other can be ignore

Attention

- **Breaking Down the Input:** Into smaller pieces, like individual words.
- **Picking Out Important Bits:** By comparing each piece to a question or 'query' it has in mind.
- **Assigning Importance:** Each piece gets a **score** based on how well it matches the question. The higher the score, the more important.
- **Focusing Attention:** After scoring each piece, it figures out how much attention to give to each one. Pieces with **higher scores** get more **attention**.
- **Putting It All Together:** Finally, it adds up all the pieces, but gives more weight to the important ones.

Attention: the core idea

- Compute a **weighted average** of relevant information
- Weights depend on the **query** (what you need)
- This gives **soft selection** instead of hard rules



Attention as retrieval

- **Query (Q)** asks: what am I looking for?
- **Keys (K)** describe: what is available?
 - research memory address
- **Values (V)** provide: what do I take?

Attention Calculation

To find the attention of a target word relative to input embeddings:

- Use the **Query (Q)** of the **target word**.
- Use the **Key (K)** of the **input word**.
- Compute a **matching score (attention weight)** from Q and K.
- Use these scores as weights to sum the Value (V) vectors.

Attention Calculation

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

Example:

To calculate the Attention score

for chasing



which previous tokens are most relevant

- take a dot product of the embedding of “chasing” with the embedding of each of the previously seen words like “The”, “FBI”, and “is”

Q / K / V in one sentence

- **Q:** representation of the current position
- **K:** representations we match against
- **V:** representations we aggregate
- **Output:** $\Sigma (\text{attention weight}) \cdot V$

Attention scoring

Target

Previous word

- $\text{score}(i,j) = \text{dot}(\mathbf{q}_i, \mathbf{k}_j)$
- Bigger score $\rightarrow$ more relevant
- **softmax over j gives weights**

Scaled dot-product attention

Scored $s_j = q \cdot k$

Weights: $a_j = \text{softmax}(s)$

Output: $z = \sum_i a_j v_j$

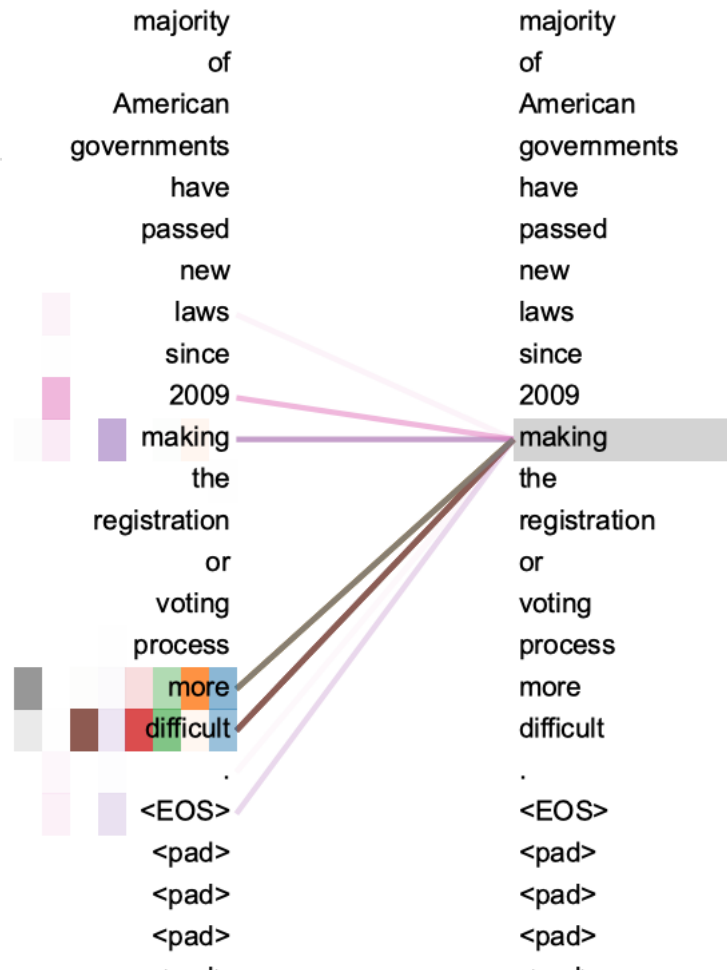
- **softmax**: weights sum to 1

Self-attention vs cross-attention

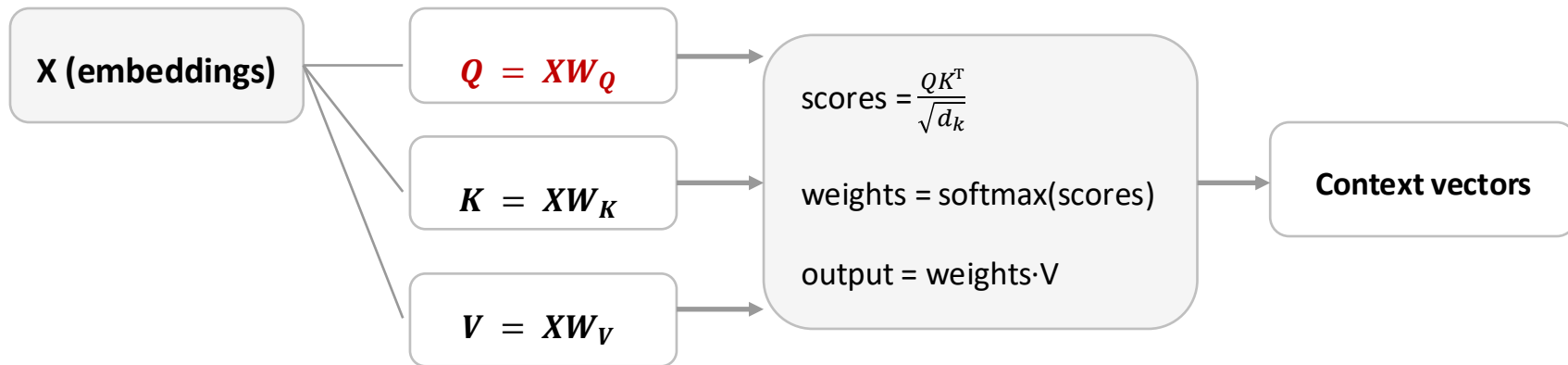
- **Self-attention**: Q,K,V from the same sequence
- **Cross-attention**: Q from decoder; K,V from encoder
- Today: focus on self-attention (Transformer core)

Self-attention

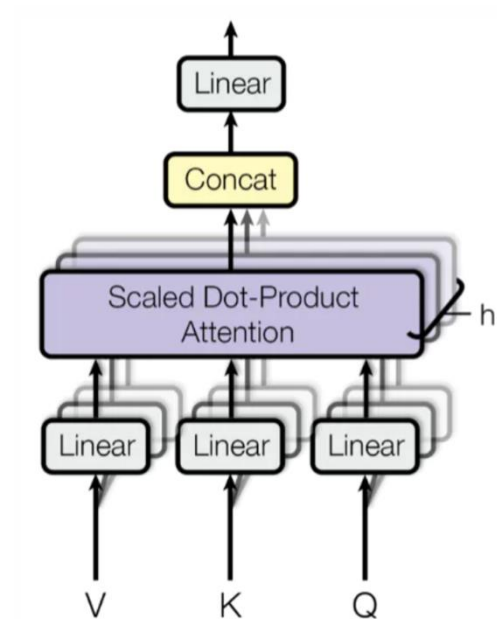
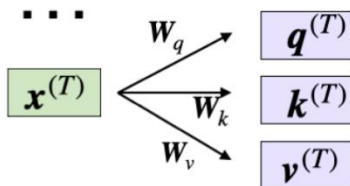
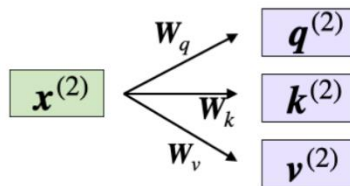
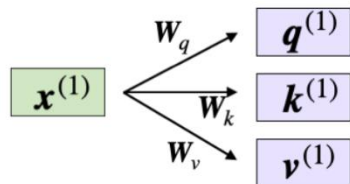
- Input: token embeddings
 $X \in R^{n \times d}$
- **Project to Q,K,V with learned matrices**
- Each position can attend to all others (global receptive field)



Compute Q / K / V

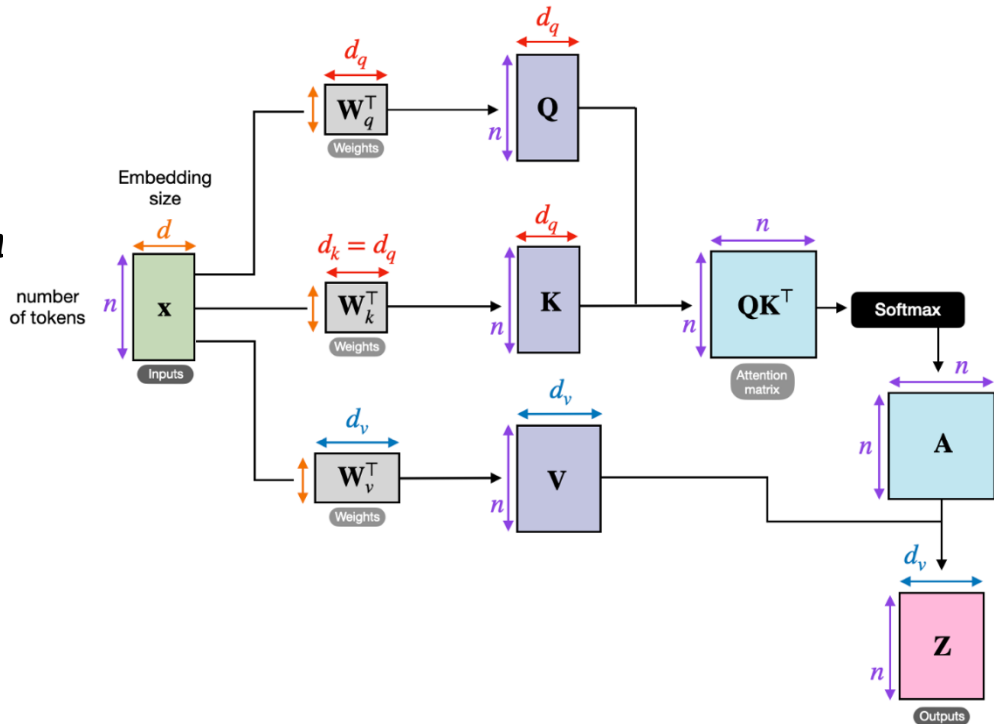


Weight Matrix



Shapes

- $X: (n \times d_{model})$
- $Q, K: (n \times d_k)$ $V: (n \times d_v)$
- **$QK^T: (n \times n)$ attention scores**
- **Output: $(n \times d_v)$**



Why scale by $\sqrt{d_k}$?

- Dot-products grow with dimension $\rightarrow$ large logits
- **softmax saturates** $\rightarrow$ tiny gradients
- **Scaling keeps scores in a reasonable range**

Next steps

- Multi-head attention, Transformer architecture
(FFN+Residual+LayerNorm)

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?